Today 10:46 AM

Lab_4_VibeCoding...
PDF

https://github.com/SETAPESU26/36_dinoRun

I checked the **Lab 4 handout** and your assigned GitHub repository.

The assignment is very straightforward: you need to **repair the existing Dino Run game and add 3 features using an LLM**, with separate commits for each task. The handout specifically says to clone the assigned repo, run it, record a 10-second video before changes, use prompts to fix/add features, record another 10-second video afterward, and push your work to your own repo.

Lab_4_VibeCoding_Student_handout

Your repo tasks

Your repository is a **Python** + **Pygame** Dino Run game. The README gives exactly **4 tasks**:

1. **Fix high-score bug**
   - The comparison in `Game.update` is backwards.
   - The high score should update when the current run beats the previous high score.
2. **Implement `dino_tint(on_ground)`**
   - Change the dino's color while it is airborne.
   - It can return an RGB tuple or `None`.
3. **Implement `on_obstacle_passed(obstacle, score)`**
   - Called once when an obstacle is successfully passed.
   - Add something like a `+1` popup or combo counter.
4. **Implement `max_jumps()`**
   - Increase the jump limit.
   - The README specifically suggests returning `2` to enable a double jump. GitHub

The repo currently contains just `game.py` and `README.md`, so this should be quite manageable.

GitHub

What I recommend we do

Don't randomly modify the code. We'll do the lab exactly in the required order:

Step 1: Clone/fork your repo
Step 2: Run the original game
Step 3: Record the required **10-second BEFORE video**
Step 4: Fix Task 1 with an LLM prompt
Step 5: Commit Task 1

Step 6: Add Task 2 → commit
Step 7: Add Task 3 → commit
Step 8: Add Task 4 → commit
Step 9: Test everything together
Step 10: Record the **10-second AFTER video**
Step 11: Push to your personal repo
Step 12: Export/save this chat history for submission

The handout also explicitly requires **separate commits for each task** and says **not to raise a PR to the main SETAPESU26 repo**. Lab_4_VibeCoding_Student_handout

If you want, we can start with **Step 1 right now**, and I'll give you the exact commands to clone the repo and run it.

give me step by strp how to do it

Absolutely. Let's do it **step by step**, and don't move to the next step until the current one works.

Your Lab 4 requires you to clone the assigned repo, run the game, record a 10-second video **before** changes, fix/add the README features using prompts, make separate commits, then record the final 10-second video and push your repo. Lab_4_VibeCoding_Student_handout

Step 1 — Clone your repository

Open **VS Code**.

Then open:

Terminal → New Terminal

Run:

```bash
git clone https://github.com/SETAPESU26/36_dinoRun.git
```

Then:

```bash
cd 36_dinoRun
```

Check the files:

Bash

```
dir
```

You should see something similar to:

Plain text

```
game.py
README.md
```

Step 2 — Open the project in VS Code

From the same terminal:

Bash

```
code .
```

Your VS Code should now open the `36_dinoRun` folder.

Step 3 — Read the README

Open:

Plain text

```
README.md
```

You should see the four tasks/features that need to be implemented:

- Fix high-score bug
- Implement `dino_tint(on_ground)`
- Implement `on_obstacle_passed(obstacle, score)`
- Implement `max_jumps()`

These are the features the lab asks you to add one by one. Lab_4_VibeCoding_Student_handout

Don't change anything yet.

Step 4 — Run the original game

First check whether Python works:

```bash
python --version
```

Then install Pygame if necessary:

```bash
pip install pygame
```

Now run:

```bash
python game.py
```

The Dino Run game should open.

If the game opens

Play it for a little while.

Do not modify the code yet.

Step 5 — Record the BEFORE video

The assignment specifically requires:

> Record 10 seconds of video before making any changes. Lab_4_VibeCoding_Student_handout

So record approximately **10 seconds of the original game**.

For example, show:

- Dino running
- Obstacles appearing
- Dino jumping
- Score changing

Save it as something like:

```plain text
```

```
before.mp4
```

Important: Do this before changing `game.py`.

Step 6 — Create your first Git commit

Before modifying anything, it's a good idea to make sure your original state is tracked.

Run:

```bash
git status
```

Then:

```bash
git add .
```

Then:

```bash
git commit -m "Initial game setup"
```

You don't need to push yet.

Step 7 — Task 1: Fix the high-score bug

Open:

```
game.py
```

Find the `Game.update` function.

You don't need to manually figure out the fix.

Since this is a **Vibe Coding lab**, give your coding AI a prompt.

Use this prompt:

```
Plain text
In game.py, fix the high-score bug described in README.md. Inspect the existing Game.update logic and make
```

Let the AI modify the code.

Then test it

Run:

```
Bash
python game.py
```

Make sure the game still works.

Step 8 — Commit Task 1

Once Task 1 works:

```
Bash
git status
```

Then:

```
Bash
git add game.py
```

Then:

```
Bash
git commit -m "Fix high score bug"
```

This is Commit 1.

Step 9 — Task 2: Implement `dino_tint(on_ground)`

Now give the AI this prompt:

Plain text

```
In game.py, implement dino_tint(on_ground) according to the README. When the dino is airborne, return an R(
```

Then run:

Bash

```
python game.py
```

Test the jump.

You should be able to see the dino change color while airborne.

Step 10 — Commit Task 2

If it works:

Bash

```
git add game.py
```

Then:

Bash

```
git commit -m "Add airborne dino tint"
```

This is Commit 2.

Step 11 — Task 3: Implement `on_obstacle_passed`

Now use:

Plain text

```
In game.py, implement on_obstacle_passed(obstacle, score) according to the README. It should be called once
```

Run:

Bash

```
python game.py
```

Play until you successfully pass an obstacle.

You should see some feedback when the obstacle is passed.

Step 12 — Commit Task 3

Bash

```
git add game.py
```

Then:

Bash

```
git commit -m "Add obstacle pass feedback"
```

This is Commit 3.

Step 13 — Task 4: Implement `max_jumps()`

Now use:

Plain text

```
In game.py, implement max_jumps() according to the README. Enable double jumping by returning 2. Make the s
```

Run:

Bash

```
python game.py
```

Test:

1. Jump once.
2. While airborne, press jump again.
3. The dino should perform a second jump.

Step 14 — Commit Task 4

Bash

```
git add game.py
```

Then:

Bash

```
git commit -m "Enable double jump"
```

This is Commit 4.

Step 15 — Check all commits

Run:

Bash

```
git log --oneline
```

You should have separate commits similar to:

Plain text

```
xxxxxxx Enable double jump
xxxxxxx Add obstacle pass feedback
xxxxxxx Add airborne dino tint
xxxxxxx Fix high score bug
xxxxxxx Initial game setup
```

This is important because your handout explicitly says:

"Have Separate Commits for each task." Lab_4_VibeCoding_Student_handout

---

Step 16 — Test the complete game

Run:

```bash
python game.py
```

Now test **everything**:

High score

Get a score and make sure the high score updates correctly.

Dino tint

Jump and check that the dino changes color in the air.

Obstacle feedback

Pass an obstacle and check for the feedback.

Double jump

Jump once and jump again while airborne.

Make sure nothing else broke.

Step 17 — Record the AFTER video

The assignment requires another **10-second video after all changes**. Lab_4_VibeCoding_Student_handout

Record approximately 10 seconds showing the modified game.

Ideally show:

- Dino running
- Jumping
- Double jump
- Changed airborne color
- Passing an obstacle

- Feedback
- Score/high score

Save it as:

```
Plain text
after.mp4
```

Step 18 — Push your code

Before pushing, check:

```
Bash
git status
```

Then:

```
Bash
git remote -v
```

Important

Your assignment says **do not raise a PR to the main `SETAPESU26` repository**. Your changes need to be pushed to **your personal repo**. Lab_4_VibeCoding_Student_handout

If you have already created your own GitHub repository, we'll change the `origin` to your repository.

For example:

```
Bash
git remote set-url origin https://github.com/YOUR_USERNAME/36_dinoRun.git
```

Then:

```
Bash
git push -u origin main
```

If your branch is called `master` instead:

Bash

```
git push -u origin master
```

Step 19 — Create the Lab-4 folder

Your submission needs a **Lab-4 folder** containing:

Plain text

```
Lab-4/
│
├── before.mp4
├── after.mp4
├── game.py
└── chat_history.pdf
```

The handout specifically lists **videos, updated code, and exported chat history** as the required deliverables. Lab_4_VibeCoding_Student_handout

---

Most important thing

Don't start all four tasks at once.

We'll do it exactly like this:

Plain text

```
STEP 1  → Clone repo
STEP 2  → Open VS Code
STEP 3  → Read README
STEP 4  → Run original game
STEP 5  → Record BEFORE video
STEP 6  → Task 1
STEP 7  → Commit Task 1
STEP 8  → Task 2
STEP 9  → Commit Task 2
STEP 10 → Task 3
STEP 11 → Commit Task 3
STEP 12 → Task 4
STEP 13 → Commit Task 4
STEP 14 → Test everything
STEP 15 → Record AFTER video
```

```
STEP 16 → Push to GitHub
STEP 17 → Add Lab-4 submission files
```

Start with Step 1 only.

Run:

Bash

```
git clone https://github.com/SETAPESU26/36_dinoRun.git
cd 36_dinoRun
code .
```

Then send me a **screenshot of your terminal/output**, and I'll tell you exactly what to do next.

give me all the 4 tasks what to do

Task 1: Fix the high-score bug

> The high score is supposed to update — and be saved to `dino_highscore.txt` — whenever the current run's score beats the previous best. In the current build, the high score never updates no matter how well you play, because the comparison that decides whether to update it is backwards. Look at the `if` condition right after a collision is detected in `Game.update`, and check which side of the comparison should be "the run that just ended" and which should be "the best score seen so far."

Task 2: Implement `dino_tint(on_ground)`

> Called once per frame in `Dino.draw`, as `color = dino_tint(self.on_ground) or self.color`. It receives a boolean: `True` while the dino is standing on the ground, `False` while it's airborne. Return an `(r, g, b)` color, or `None` to keep the default green. Idea: give the dino a different color while it's mid-jump.

Task 3: Implement `on_obstacle_passed(obstacle, score)`

> Called from `Game.update` exactly once per obstacle, the first frame that obstacle's right edge scrolls past the dino's left edge (i.e. it's been successfully jumped, not just spawned or removed off-screen). It receives the `Obstacle` that was passed and the score at that moment. Its return value is ignored. Idea: a small "+1"

popup, or a running combo counter for consecutive obstacles cleared without a hit.

Task 4: Implement `max_jumps()`

> Called every time the player tries to jump, as part of `if self.jump_count < (max_jumps() or 1):` in `Dino.jump`. It takes no arguments and should return an integer — the number of jumps the dino gets before it must touch the ground again — or `None` for the default of 1 (no double jump). The `jump_count` bookkeeping (incrementing on jump, resetting to 0 on landing) is already implemented; you only need to raise the limit. Idea: return `2` for a double jump.

---

these are my 4 tasks

Yes — those are the **exact 4 tasks** you need to implement. Here is what you should do for each one, including the prompt to give your Vibe Coding tool and what you should verify afterward.

Task 1 — Fix the High-Score Bug

What you need to do

The current code has the high-score comparison backwards.

Find this section in:

Plain text

```
game.py
```

inside:

Plain text

```
Game.update
```

specifically the `if` condition immediately after a collision is detected.

The logic should be:

Python

```
if current_score > high_score:
```

not:

```python
if high_score > current_score:
```

The purpose is:

```
Current run score
       ↓
Is it greater than
       ↓
Previous high score?
       ↓
YES → update high score
       ↓
save to dino_highscore.txt
```

Prompt to use

```
Fix Task 1 from the README. In Game.update, find the if condition immediately after a collision is detecte
```

Test

Play the game and get a score higher than the existing high score.

Then check:

```
dino_highscore.txt
```

The new high score should be saved there.

Commit

```bash
```

```
git add game.py dino_highscore.txt
git commit -m "Fix high score bug"
```

Task 2 — Implement `dino_tint(on_ground)`

What you need to do

There is already a function:

```python
dino_tint(on_ground)
```

It is called every frame by:

```python
color=dino_tint(self.on_ground) or self.color
```

The argument means:

```
on_ground=True
        ↓
Dino is standing

on_ground=False
        ↓
Dino is airborne
```

You need to make the dino a **different color while jumping**.

For example:

```python
def dino_tint(on_ground):
    if not on_ground:
        return (255, 200, 0)
    return None
```

This means:

Plain text

```
Ground → None → normal green dino
Airborne → RGB color → different colored dino
```

Prompt to use

Plain text

```
Implement Task 2 from the README. Implement dino_tint(on_ground) so that the dino has a different RGB colo
```

Test

Run:

Bash

```
python game.py
```

You should see:

Plain text

```
Dino on ground → normal color
       ↓
Jump
       ↓
Dino changes color
       ↓
Land
       ↓
Dino returns to normal color
```

Commit

Bash

```
git add game.py
git commit -m "Add airborne dino tint"
```

Task 3 — Implement `on_obstacle_passed(obstacle, score)`

This one is slightly more interesting.

What you need to do

The function:

```python
on_obstacle_passed(obstacle, score)
```

is already called by the game.

You **do not need to change when it gets called**.

The README says it is called exactly once when the obstacle successfully passes the dino.

You need to add some visible feedback.

The easiest option is a small:

```
+1
```

popup.

For example:

```
      +1
🦕          ▓▓
```

when the dino successfully clears an obstacle.

You could also implement a combo counter, but **don't make this unnecessarily complicated**.

Prompt to use

```
Implement Task 3 from the README. Implement on_obstacle_passed(obstacle, score) with a simple visible feedb
```

Test

Run:

```bash
python game.py
```

Jump over an obstacle.

When you successfully pass it, you should see something like:

```
+1
```

The important part is that it happens **when the obstacle is passed**, not when:

- the obstacle spawns
- the obstacle is removed
- the player collides with it

Commit

```bash
git add game.py
git commit -m "Add obstacle pass feedback"
```

Task 4 — Implement `max_jumps()`

This is the easiest one.

What you need to do

The existing code already contains:

```python
if self.jump_count < (max_jumps() or 1):
```

The game already handles:

```
jump_count += 1
```

and resets the count when the dino lands.

You only need to change the maximum number of jumps.

Currently the default is:

```
Plain text
```

```
1 jump
```

You need:

```
Plain text
```

```
2 jumps
```

So:

```
Python
```

```
def max_jumps():
    return 2
```

This enables a **double jump**.

Prompt to use

```
Plain text
```

```
Implement Task 4 from the README. Make max_jumps() return 2 so the dino can perform a double jump. Do not r
```

Test

Run:

```
Bash
```

```
python game.py
```

Then:

Plain text

```
Press jump
    ↓
Dino goes into air
    ↓
Press jump again
    ↓
Dino performs second jump
```

You should **not** be able to jump a third time before landing.

Commit

Bash

```
git add game.py
git commit -m "Enable double jump"
```

Your complete 4-task sequence

Do them **one at a time**:

Task	What you change	What you test	Commit
1	Fix `current_score > high_score` comparison	High score saves correctly	`Fix high score bug`
2	Implement `dino_tint()`	Dino changes color in air	`Add airborne dino tint`
3	Implement `on_obstacle_passed()`	`+1` appears after clearing obstacle	`Add obstacle pass feedback`
4	`max_jumps()` returns `2`	Double jump works	`Enable double jump`

Important

For your lab, **don't combine the four tasks into one prompt or one commit**.

You want your Git history to clearly show:

Plain text

```
Task 1 → commit
Task 2 → commit
Task 3 → commit
Task 4 → commit
```

That directly satisfies the requirement for separate commits for each task.

Lab_4_VibeCoding_Student_handout

And for the Vibe Coding requirement, **save these exact prompts and the AI responses** because your final submission also requires the exported chat history. Lab_4_VibeCoding_Student_handout

game.py
Code

this is my game.py

Yes. Now that I can see your **actual** `game.py`, I can tell you exactly what each task needs to change.

game

Your file is intentionally set up with the four tasks unfinished using `pass`:

- `dino_tint()` → line 13
- `on_obstacle_passed()` → line 18
- `max_jumps()` → line 23
- High-score condition → line 144

So let's do them **one by one**, without changing anything unnecessary.

Task 1 — Fix High Score

Your current code

At lines 141–146 you have:

Python

```
if hit:
    self.state = "lose"
    current = self.score // 10
    if self.high_score > current:
        self.high_score = current
        save_high_score(self.high_score)
    return
```

The problem is this:

Python

```
if self.high_score > current:
```

It's backwards.

You want:

Python

```
if current > self.high_score:
```

Because:

Plain text

```
current = score from the run that just ended
self.high_score = previous best
```

You only update when:

Plain text

```
current > previous best
```

Give your Vibe Coding tool this prompt

Plain text

```
Fix Task 1 from the README in game.py. In Game.update, immediately after a collision is detected, the high·
```

Expected change

Python

```
if current > self.high_score:
```

That's it.

Test

Run:

Bash

```
python game.py
```

Play until you lose.

If your score is higher than the previous best, `dino_highscore.txt` should contain the new score.

Commit

Bash

```
git add game.py
git commit -m "Fix high score bug"
```

Task 2 — `dino_tint(on_ground)`

Your current function is:

Python

```
def dino_tint(on_ground):
    """Return an (r, g, b) colour override for the dino based on whether it's on the ground, or None for th
    pass
```

And your `Dino.draw()` already does:

Python

```
color = dino_tint(self.on_ground) or self.color
```

So **you don't need to modify `Dino.draw()`**.

You only need to implement `dino_tint()`.

What it should do

When:

Python

```
on_ground=True
```

return:

Python

```
None
```

so the normal dino color is used.

When:

Python

```
on_ground=False
```

return an RGB color.

For example:

Python

```
def dino_tint(on_ground):
    if not on_ground:
        return (255, 180, 50)
    return None
```

The exact color isn't specified, so the AI can choose one.

Prompt

Plain text

```
Implement Task 2 from the README in game.py. Implement dino_tint(on_ground) so that the dino has a differer
```

Test

Run:

Bash

```
python game.py
```

Press SPACE to jump.

You should see:

Plain text

```
GROUND → normal green
JUMP   → different color
LAND   → normal green
```

Commit

Bash

```
git add game.py
git commit -m "Add airborne dino tint"
```

Task 3 — `on_obstacle_passed(obstacle, score)`

Your current function is:

Python

```
def on_obstacle_passed(obstacle, score):
    """Called once, the frame an obstacle finishes scrolling past the dino. Add a sound or a combo counter
    pass
```

And importantly, your existing `Game.update()` already calls it here:

Python

```
if not obs.passed and obs.rect.right < self.dino.rect.left:
    obs.passed = True
    on_obstacle_passed(obs, self.score // 10)
```

So **do not change this logic**.

It already guarantees the function is called once per obstacle.

You only need to implement the function.

One important thing

Your function needs to create something **visible** if you want to demonstrate it in your video.

A simple `+1` popup is good.

However, your current game doesn't have any popup system, so asking the AI to add a small popup will require it to introduce a little supporting state/drawing logic.

That's okay.

Prompt

Use this:

Plain text

```
Implement Task 3 from the README in game.py. The existing Game.update code already calls on_obstacle_passe
```

Test

Run:

Bash

```
python game.py
```

Jump over an obstacle.

You should get something like:

Plain text

```
      +1
       🦖        ▌
```

The important thing is:

Plain text

```
Obstacle successfully passed
          ↓
        +1 popup
```

Not:

Plain text

```
Obstacle spawned → +1
```

and not:

Plain text

```
Collision → +1
```

Commit

Bash

```
git add game.py
git commit -m "Add obstacle pass feedback"
```

Task 4 — `max_jumps()`

Your current function is:

Python

```
def max_jumps():
    """Return how many jumps the dino gets before it must land again (2 for a double jump), or None for the
    pass
```

And your existing `Dino.jump()` is:

Python

```
def jump(self):
    if self.jump_count < (max_jumps() or 1):
        self.vel_y = JUMP_FORCE
        self.on_ground = False
        self.jump_count += 1
```

The jump bookkeeping is **already done for you**.

You only need to return `2`.

Implementation

Python

```
def max_jumps():
    return 2
```

This gives:

```
Plain text

First SPACE
    ↓
jump_count = 1

Second SPACE while airborne
    ↓
jump_count = 2

Third SPACE
    ↓
NOT allowed
```

Prompt

```
Plain text

Implement Task 4 from the README in game.py. Make max_jumps() return 2 so the dino can perform a double jur
```

Test

Run:

```
Bash

python game.py
```

Then:

1. Press SPACE.
2. While still in the air, press SPACE again.
3. The dino should jump again.
4. Press SPACE a third time while airborne.
5. Nothing should happen.
6. After landing, you should be able to jump twice again.

Commit

Bash

```
git add game.py
git commit -m "Enable double jump"
```

Your final code changes

After completing all four tasks, the important parts should conceptually look like this:

Task 1

Python

```
if current > self.high_score:
```

Task 2

Something like:

Python

```
def dino_tint(on_ground):
    if not on_ground:
        return (255, 180, 50)
    return None
```

Task 3

A visible popup/feedback implementation inside:

Python

```
def on_obstacle_passed(obstacle, score):
```

Task 4

Python

```
def max_jumps():
    return 2
```

One important correction to my previous answer

Now that I've actually seen your file, **Task 3 is the only one that may require more than a tiny one-function change**, because your current code has no existing popup state/rendering system. So let the Vibe Coding tool implement the smallest supporting mechanism needed for the `+1` feedback.

For Tasks **1, 2, and 4**, the changes should be extremely small.

And don't manually implement everything before using the Vibe Coding prompts — your lab specifically asks you to use prompts to fix/add the features. Lab_4_VibeCoding_Student_handout

I recommend starting with Task 1 now. Give the AI the Task 1 prompt above, let it modify `game.py`, and then send me the resulting code/diff. I'll check whether it changed **only what it should** before you commit it.

Today 6:44 PM

task 3 doesnt seem right

You're right to question it. Looking at your **actual `game.py`**, Task 3 needs to be handled more carefully.

The README says the function is already called **exactly once** when the obstacle passes the dino, and its return value is ignored. game

The task is:

> `on_obstacle_passed(obstacle, score)` — add something like a small `+1` popup **or** a running combo counter.

So the important point is: **the function itself should provide the feature**, but your current game has no existing popup mechanism. My previous prompt asking the AI to build a popup system was unnecessarily complicated.

The simpler approach

A **combo counter** is much more appropriate for this code.

You can add a global counter:

```python
combo=0
```

Then:

Python

```
def on_obstacle_passed(obstacle, score):
    global combo
    combo+=1
```

But there's a problem: **nothing currently draws `combo` on the screen**, so the user wouldn't see anything.

A better implementation is to add a small piece of game state to `Game`, but that means modifying more than just the function.

Even simpler: use the existing score

Since the function receives:

Python

```
obstacle, score
```

the intended task may simply be to add some feedback such as a sound or counter, as the function's own docstring says:

Python

```
"""Called once, the frame an obstacle finishes scrolling past the dino. Add a sound or a combo counter here
```

game

So I'd use a **sound effect** instead of creating a popup system.

For example, `on_obstacle_passed()` could trigger a simple Pygame sound — but your current file doesn't load or create any sound, so we'd again have to add supporting code.

What I would do for your lab

Ask the Vibe Coding tool to implement **the smallest visible feature possible**, while preserving the existing structure:

Plain text

```
Implement Task 3 exactly as described in README.md and the existing docstring. The function on_obstacle_pas
```

That is much closer to the task than my earlier `+1 popup` suggestion.

One thing to watch

Your current call is:

Python

```
on_obstacle_passed(obs, self.score // 10)
```

So the `score` argument is already the **current displayed score**, not the raw frame score. game

And this:

Python

```
if not obs.passed and obs.rect.right < self.dino.rect.left:
```

already guarantees the callback happens once per obstacle because immediately afterward:

Python

```
obs.passed=True
```

So **do not let the AI rewrite that part**.

If you want, I can also tell you **exactly what Task 3 should look like in your `game.py`, line by line**, without unnecessarily complicating your game.

fine i understood the logic i will try on my own to do it

Yep 👍 That's actually the best way to do this lab—you understand the logic first, then implement it yourself.

For **Task 3**, just keep these three things in mind:

1. `on_obstacle_passed()` is already called at the correct moment.
2. It is called **only once per obstacle** because `obs.passed` becomes `True`.
3. Your job is only to add some visible/useful behavior there, such as a combo counter or `+1` feedback.

Try it yourself first. If you get an error or your implementation doesn't behave as expected, send me your code and I'll help you fix **only that part**.

ok i got of a logic but i am not able to code it
can you tell what to code

i want to use popups on top and start incrementing it

Yes — your logic can be very simple:

Every time an obstacle is successfully passed → increment a combo counter → show the counter at the top.

Since `on_obstacle_passed()` is already called exactly once per obstacle, you don't need to change the obstacle logic. ‹/› game

1. Add a combo variable

Inside `Game.reset()`, add:

‹/› Python

```
self.combo=0
```

This is important because when a new game starts, the combo should go back to `0`.

Your `reset()` becomes:

‹/› Python

```
def reset(self):
    self.dino=Dino(80,GROUND_Y)
    self.obstacles=[]
    self.score=0
    self.combo=0
    self.speed=6
    self.spawn_timer=0
    self.spawn_interval=90
    self.state="wait"
```

2. Increment it when an obstacle is passed

Here's the tricky part: your current function is **outside the `Game` class**:

‹/› Python

```
def on_obstacle_passed(obstacle,score):
    pass
```

So it doesn't directly have access to `self.combo`.

The cleanest solution is to move the counter into the `Game` class, but your existing function signature is intentionally provided by the assignment.

For your current structure, I would instead make the function maintain its own counter:

```python
combo=0

def on_obstacle_passed(obstacle,score):
    global combo
    combo+=1
```

3. Display it at the top

In `Game.draw()`, you already have:

```python
score_surf=self.font.render(f"Score: {self.score // 10}   Best: {self.high_score}",True,(50,50,50))
screen.blit(score_surf,(WIDTH-260,20))
```

Add another text render below/near it:

```python
combo_surf=self.font.render(f"Combo: {combo}",True,(50,50,50))
screen.blit(combo_surf,(20,20))
```

So visually you get something like:

```
Combo: 0                              Score: 12   Best: 20


                 🦖        ▮
```

Then:

```
Combo: 1                              Score: 13   Best: 20
```

then:

Plain text

```
Combo: 2                          Score: 14   Best: 20
```

and so on.

One issue with this approach

Because `combo` is global, it **won't automatically reset when `Game.reset()` is called**.

So if you use this approach, you need to reset it when a new game starts:

Python

```
global combo
combo=0
```

inside `reset()`.

That works, but I actually recommend a cleaner approach for your game: **make `combo` a `Game` attribute** and have the callback return the new value, then let `Game.update()` store it. But that requires slightly changing the existing callback usage.

Since this is your lab and you're trying to implement the logic yourself, **I'd start with the global version above because it is the smallest change and directly demonstrates your idea.**

i did it little different
i did popup +1 but i am not able to display it
this is the line i wrote
popups.append([obstacle.rect.centerx, obstacle.rect.top -10,30])

Yes — your line is actually a good start:

Python

```
popups.append([obstacle.rect.centerx, obstacle.rect.top-10,30])
```

You're storing:

Plain text

```
[x position, y position, timer]
```

The missing part is that **`append()` only stores the popup data. It doesn't draw anything on the screen.**

You need to update and draw the popup inside `Game.draw()`.

1. Create the popup list

You need this somewhere before `on_obstacle_passed()` is called:

```python
popups=[]
```

A good place is near your other global variables.

---

2. Your function

Your function can remain roughly:

```python
def on_obstacle_passed(obstacle,score):
    popups.append([obstacle.rect.centerx,obstacle.rect.top-10,30])
```

The `30` is your popup lifetime.

---

3. Draw the popup

Inside `Game.draw()`, after drawing the obstacles, add:

```python
for popup in popups:
    popup[1]-=1
    popup[2]-=1
    popup_surf=self.font.render("+1",True,(50,50,50))
    screen.blit(popup_surf,(popup[0],popup[1]))
```

Now the `+1` should actually appear.

---

4. Remove expired popups

You don't want them to stay forever.

After the drawing loop, add:

Python

```
popups[:]=[p for p in popups if p[2]>0]
```

So your popup section becomes:

Python

```
for popup in popups:
    popup[1]-=1
    popup[2]-=1
    popup_surf=self.font.render("+1",True,(50,50,50))
    screen.blit(popup_surf,(popup[0],popup[1]))

popups[:]=[p for p in popups if p[2]>0]
```

What happens now

When an obstacle passes:

Plain text

```
obstacle passed
      ↓
on_obstacle_passed()
      ↓
popups.append([x,y,30])
      ↓
Game.draw()
      ↓
draw "+1"
      ↓
move popup upward
      ↓
timer decreases
      ↓
timer = 0
      ↓
popup disappears
```

So your original idea is correct. You were just **storing the popup but never rendering it**.

One small improvement: since you're putting the popup at `obstacle.rect.top-10`, the obstacle may move away immediately after the callback. If you want the `+1` to stay visually where it appeared, storing the coordinates like you're doing is exactly the right approach.